

Recovering Semantics from Unknown Binaries with Known Input Formats

Anonymous
Anonymous Institution

Abstract—Reverse engineers who already understand the input format a binary parses spend most of their initial effort locating the code that implements each piece of the format. We present ParseRE, a binary-only tool that automates that step. Given a grammar template for the input format, ParseRE instantiates a corpus of structurally varied inputs, records QEMU translation logs, compares pairs whose parse trees differ in exactly one production, and attributes differential trace-derived edges to that production. A two-step uniqueness filter removes shared evidence, and the result is a trace-derived graph whose surviving records carry candidate grammar-rule labels. The prototype requires neither source code nor compiler instrumentation, although its translation-log tracer does not capture a guaranteed complete dynamic execution sequence. The artifact contains completed runs for Curl URI parsing, json-c, and libelf, plus planned evaluations for Apache APR and HPACK. On the completed ELF evaluation against libelf, manual review yields 62 strict true positives among 68 labeled basic blocks (91.2%), with one additional marginal case. Accuracy review for the committed URI and JSON outputs remains future work.

Index Terms—reverse engineering, binary analysis, control-flow graphs, parser analysis, input format recovery, QEMU

1. Introduction

Modern reverse-engineering workflows begin with a binary, a vague sense of what it does, and a long road from there to understanding. When the binary parses a known input format, much of that road can be skipped: any reverse engineer who already understands the format spends most of their initial effort locating the code that implements each piece of it. Existing tools either annotate the input (PolyFile [1]), require source-level instrumentation to track data flow through the parser (PolyTracker [2]), or compare two execution traces by hand (Tenet [3]). They do not directly address the narrower task studied here: associating known grammar productions with locations in a runnable binary using corpus-level translation-log differences.

The Core Idea. We present ParseRE, a binary-only labeling tool driven by parse-tree differencing. Given a grammar template for the target’s input format,

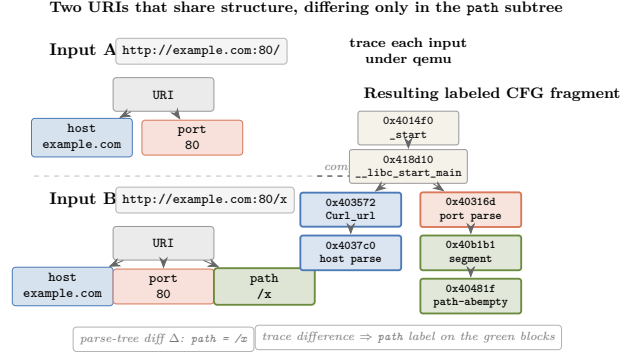


Figure 1. Two URIs that share scheme, host, and port but differ in the path subtree. A small set of parser block records differs between the runs; surviving differential records become candidates for the path label. ParseRE applies this idea across a structured corpus.

ParseRE instantiates a corpus of structurally varied inputs, records each run under QEMU user-mode emulation, and compares pairs whose parse trees differ in exactly one production. Figure 1 sketches the idea on two URIs that differ only in their path subtree: translation-log records unique to Input B can be associated with the path production. Aggregated over the full corpus and filtered for uniqueness, the result is a labeled trace-derived graph whose nodes correlate with grammar rules.

Why this works in practice. Parsers tend to dispatch on grammar production at the function level. A URI parser has a separate routine for host validation than for path canonicalization. A JSON parser has separate code paths for numeric tokens, string tokens, and structural tokens. Two inputs that differ only in their path subtree may therefore differ in the code translated while processing paths. ParseRE uses differences between consecutive translation-log record pairs as candidate evidence for the production that varied.

Why this is hard in practice. Real parsers also share large amounts of code across productions, including memory allocation, error reporting, and common token tape advancement. Naively attributing every differential edge to a production produces a noisy label set in which most labels appear on most blocks. ParseRE applies a two-step uniqueness filter that strips record pairs shared

across multiple productions before reporting a label. In the manually reviewed libelf run, 62 of 68 reported labels are strict true positives.

Evaluation. The current artifact contains runs for URI (parsed by Curl), JSON (parsed by json-c), and ELF (parsed by libelf), while Apache APR and HPACK evaluations remain planned. The ELF output has been manually reviewed and yields 62 strict true positives among 68 labeled blocks (91.2%); URI and JSON accuracy review remains incomplete.

Contributions.

- We describe a binary-only technique that transfers semantic labels from program inputs onto nodes and trace-derived edges (Section 3).
- We implement the technique in ParseRE, a Python research prototype that requires a target binary and grammar template, and provide a source-available artifact for review.
- We report completed artifact runs on Curl, json-c, and libelf, including a manual review of the ELF labels (Section 5).
- We document how uniform record readers can leave some binary-format productions unlabeled when input changes do not alter the observed translation-log evidence.

2. Background

2.1. Control-Flow Graphs

Reverse engineering is the process of analyzing a binary to understand its structure, functionality, and behavior without access to source code. A fundamental program representation in this domain is the control-flow graph (CFG). Each node in a CFG corresponds to a basic block: a maximal sequence of consecutive instructions with a single entry point and a single exit point [4]. If execution reaches the first instruction of a basic block, it will execute every instruction in that block before transferring control elsewhere. Edges in a CFG represent transfers of control flow, including branches, function calls, and returns.

Understanding how a program’s input drives its execution path through the CFG is crucial to understanding the program’s behavior. For a parser binary, different inputs can exercise different code paths, so input structure can be correlated with observed execution evidence.

2.2. Execution Tracing with QEMU

QEMU [5] is an open-source emulator that supports both full-system and user-mode emulation across many architectures. In user-mode emulation, QEMU executes a single Linux binary by translating its instructions and forwarding system calls to the host kernel.

A useful feature for binary analysis is QEMU’s logging facility, which can record each translation block

when QEMU translates it, including the block’s start address and disassembly. The current prototype uses `-d in_asm` and treats the resulting translation-log order as trace-derived control-flow evidence without modifying the target binary. Because QEMU may cache and reuse translated blocks, this log is not guaranteed to be a complete dynamic execution sequence; Section 4.2 discusses the resulting limitation.

This capability is central to our approach. We run the target under QEMU for many inputs and compare the resulting logs to infer which code regions correlate with which input features.

2.3. Parse Trees and Input Annotations

A parse tree represents the syntactic structure of an input according to a grammar. Internal nodes correspond to grammar productions; leaves hold the concrete bytes.

We use parse trees to annotate inputs: each subtree is labeled with the grammar production it instantiates. When two inputs share the same tree structure except at one subtree, differences in their observed translation-log records become candidate evidence for the differing production. This correlation is the core hypothesis behind ParseRE.

3. Design

ParseRE runs as a four-stage pipeline, shown in Figure 2. The top row converts a grammar template into per-input translation logs. The bottom row converts those logs into a labeled trace-derived graph. Every stage except the grammar template itself is format-agnostic; adding a new format requires only writing a new template and compiling a small harness against the parser library under test.

3.1. Corpus Generation

A grammar template is a tree of production rules. Each leaf carries a small set of byte-level alternatives; each internal node carries a set of child configurations. ParseRE enumerates every combination of alternatives through Cartesian product, producing a corpus of concrete inputs each paired with its parse tree.

The templates are deliberately small. The JSON template covers objects, arrays, strings, numbers, booleans, null, and a handful of degenerate cases (nan, inf), instantiating to 27 inputs. The URI template covers scheme, authority, path, query, and fragment, instantiating to 1,458 inputs. The ELF template, described in Section 5.5, instantiates to 20 inputs.

The key property. For every pair of inputs we generate, the parse trees differ in either a single production or a small set of productions. This is what makes attribution possible. If two inputs differed in arbitrary ways, the trace differences between them could not be cleanly attributed to any single grammar element.

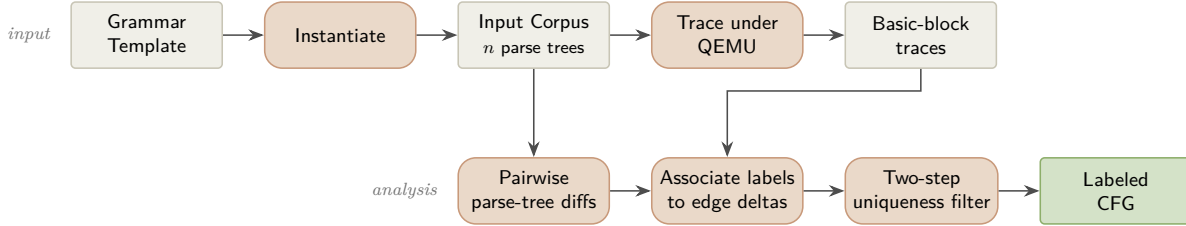


Figure 2. The ParseRE pipeline. Cream rectangles are data artifacts. Salmon rounded rectangles are processing steps. The green rectangle is the final output. Vertical arrows bridge the two rows: the input corpus drives translation logging and pairwise input differencing, while the resulting block records feed label association.

3.2. Execution Tracing

Each input is fed to the target binary, which runs under QEMU in user-mode emulation. QEMU logs translation blocks to a per-input file with each block’s start address and disassembly. ParseRE parses the records into an ordered sequence, retains only blocks within the target binary’s executable segment, and converts consecutive records into a trace-derived edge map. These record pairs are useful differential signals, but the -d in `_asm` translation log does not guarantee that every pair is a dynamic CFG edge.

Logs from inputs that the harness rejects (non-zero exit status) are dropped so the graph reflects only accepted corpus members.

3.3. Label Association

For every pair of inputs (T_i, T_j) whose parse trees differ in exactly one production p , ParseRE computes the consecutive-record-pair delta $E_i \setminus E_j$ and associates those pairs with p . Aggregating over every input pair that isolates p yields a per-production evidence set.

Two-step uniqueness filter. Raw edge-to-label associations are noisy. Many edges are shared across productions, including error-handling code and shared token-tape advancement. ParseRE strips this noise in two steps:

- 1) **Subset removal.** If the edge set for production p_1 is nearly a subset of the edge set for p_2 ($|p_1 \setminus p_2|/|p_1| < 0.1$), ParseRE removes p_1 ’s edges from p_2 ’s set. This eliminates setup and teardown code that runs for multiple productions but is most strongly associated with one.
- 2) **Strict uniqueness.** For every production, ParseRE subtracts the edges attributed to every other production. Edges that survive belong to a single production and receive its label.

After filtering, ParseRE propagates colors backward through the trace-derived graph. A node whose outgoing edges all carry the same label is itself labeled with that color, which captures basic blocks that serve as sole entry points into a production-specific subgraph.

3.4. Output

ParseRE emits three artifacts: a Graphviz DOT file of the labeled trace-derived graph, a rendered SVG for browser inspection, and an address-to-label mapping. The mapping can be adapted for import into a disassembler, but this artifact does not yet include a dedicated import script.

4. Implementation

ParseRE is implemented as a single Python module of around 850 lines, with five external dependencies (networkx, pydot, pyelftools, tqdm, cxxfilt) and a parallel work pool that scales to all available cores. Running ParseRE requires Python 3.13 and a QEMU user-mode build for the target architecture. The analyst supplies the target binary, a grammar template for the input format, and optionally a path to `addr2line` for source-line attribution during manual evaluation.

4.1. Corpus Generation

Grammar templates are written as nested data structures. Each template node carries a production name and a set of alternatives. At leaves, alternatives are concrete byte sequences. At internal nodes, alternatives are ordered tuples of child templates. ParseRE recursively enumerates every combination through Cartesian product, producing a set of concrete parse trees that serialize to byte strings.

Branching factor sets corpus size. The implementation compares ordered pairs of distinct inputs, so N instantiations produce $N(N - 1)$ comparisons. The JSON template yields 27 instantiations (702 comparisons), the URI template yields 1,458 instantiations (2,124,306 comparisons), and the ELF template yields 20 instantiations (380 comparisons). The text-format templates are larger because the underlying grammars have more independently varying productions. Each input remains small (under one kilobyte for the text formats, exactly 1,048 bytes for ELF), which keeps trace collection practical even when the input count grows.

Adding a new format. A new format requires two pieces: a template definition and a harness binary that exercises the parser library under test. The template plugs into the existing format dispatch. The harness compiles into the Docker image alongside the existing harnesses. No changes to the core pipeline are needed.

4.2. Tracing with QEMU

Each input is fed to the target via standard input. ParseRE invokes QEMU with `-d in_asm` and a per-input output path, parses the resulting translation log with a regular expression that matches QEMU’s block records, and constructs a basic-block object per record. Only blocks within the target binary’s executable segment are retained; blocks from the dynamic linker and the QEMU runtime itself are discarded by checking each address against the target’s `PT_LOAD` program header.

Logs for inputs with non-zero exit status are excluded so the constructed graph reflects accepted corpus members.

Tracing limitation. `-d in_asm` reports blocks when translated, and QEMU’s translation cache may suppress records for later executions of an already translated block. The current adjacency construction therefore captures order in the translation log, not a guaranteed complete sequence of dynamic control transfers. The reported associations should be interpreted as differential, trace-derived evidence. A stronger implementation would collect execution events with QEMU’s TCG plugin interface and validate each edge against a conventional dynamic CFG.

4.3. Target Harnesses

For each parser under test we compile a small C harness that links against the parser library, reads input from standard input, invokes the top-level parse function, and returns the result. Harnesses are typically twenty to thirty lines. The ELF harness is longer (around one hundred lines) because `libelf` exposes the parser as a family of small calls (one per program header, one per section header, one per symbol) rather than a single top-level entry point, and the harness must call all of them to exercise the full structure.

Static linking is required by the current address filter. ParseRE retains blocks only when their addresses fall inside the target executable’s selected executable `PT_LOAD` segment. With a dynamically linked harness, parser-library code lives in a separate shared object and is filtered out, which caused the initial URL run to produce no useful labels. Rebuilding `curl` from source with `-without-shared` placed its parser code inside the harness executable and restored the differential signal. Every harness in the current evaluation links statically (`libelf` via the `-static` GCC flag, `json-c` via its `.a` archive from a CMake build, `curl` via its source-built static archive).

Compilation flags. All harnesses compile with `-O0 -fno-inline` so the compiler does not collapse distinct parser functions into shared inlined code, which would destroy the correspondence between grammar productions and basic-block ranges. We also use `-no-pie` to keep addresses stable so that `addr2line` resolves cleanly against the binary on disk, and `-g` for debug symbols.

4.4. Supported Formats

The current implementation includes grammar templates for four formats:

- URI (regular): scheme, authority (userinfo, host, port), path, query, fragment, per RFC 3986.
- JSON (context-free): objects, arrays, strings, numbers, booleans, null, per RFC 8259.
- HPACK (context-sensitive binary): planned support for indexed headers, literal headers, Huffman strings, and variable-length integers, per RFC 7541. The current artifact does not contain a completed HPACK run.
- ELF (offset-driven binary): one fixed ELF header, four program-header variants, five section variants. Each instantiation is a valid 1,048-byte ELF64 file constructed by a Python generator that keeps the inter-field offsets consistent across all twenty corpus members.

5. Evaluation

The current artifact contains completed runs for three format-parser pairs: URI with `Curl`, JSON with `json-c`, and ELF with `libelf`. Manual per-block review is complete for ELF; URI and JSON results are descriptive counts from the committed outputs. Apache APR and HPACK are retained as planned extensions, not completed evaluations.

Research questions.

- (Q1) How many basic blocks does ParseRE label, and what fraction of the trace-derived graph do those labels cover?
- (Q2) For the reviewed ELF output, how accurate are the labels at the per-block level?
- (Q3) What preliminary differences appear across the three completed format-parser runs?

5.1. Methodology

For each completed format-parser pair, we (1) compile a harness against the parser library inside a Docker container with debug symbols and static linkage, and (2) run ParseRE with the corresponding grammar template under QEMU user-mode. For ELF, a human evaluator additionally checks each production-labeled block against the source location recovered via `addr2line` and classifies it as a strict true positive, false positive, or

marginal case. Because the evaluation does not enumerate all semantically relevant parser blocks independently of ParseRE, it does not measure false negatives.

Why manual inspection. Automated semantic verification without ground truth can introduce circularity or unsupported judgments. Accordingly, the ELF accuracy result comes from manual inspection of the labeled graph, source mappings, and relevant parser routines. URI and JSON accuracy values are intentionally left unreported until the same review is complete.

5.2. URI Parsers

The completed URI run targets Curl’s `curl_url_set`; a run against Apache Portable Runtime’s `apr_uri_parse` remains planned. The URI template instantiates 1,458 inputs and produces 2,124,306 ordered comparisons between distinct inputs.

Curl. On the statically linked Curl harness, ParseRE produces a trace-derived graph with 522 nodes and 558 edges. The committed mapping contains 191 production-labeled blocks across 10 exact label strings and 21 blocks with the ambiguous None label. Table 1 shows the distribution. Host validation dominates because Curl’s parser performs extensive checks: IPv4 versus IPv6 detection, IDNA processing, bracket handling, and length validation. The duplicate-looking labels (port versus `:port`, query versus `?query`) come from the template’s leaf naming: the colon or question mark delimiter is included in the label string. This is a template quirk; the underlying attribution is correct.

TABLE 1. URI label distribution (Curl).

Production	Labeled BBs
host	91
port	34
segment	30
userinfo	25
query	5
fragment	5
path	1
ambiguous	21

The 21 ambiguous blocks are code paths that the current filtering could not uniquely attribute to a single production. Per-block accuracy has not yet been reviewed.

Apache APR. The repository documents this as a planned replication with the same URI template, but it contains no APR output from which to report results.

5.3. JSON Parsers

We evaluate on `json-c`, a widely deployed C library for JSON parsing. The JSON template instantiates 27 inputs and produces 702 ordered comparisons between distinct inputs.

On `json-c` (commit 894856), ParseRE produces a trace-derived graph with 464 nodes and 400 edges, of which 295 basic blocks receive production labels across 6 productions. Table 2 shows the distribution.

TABLE 2. JSON label distribution (`json-c`).

Production	Labeled BBs
number	110
object-element	77
string	39
array	36
boolean	24
nan	9

number dominates because C-level numeric parsing is complex: sign handling, decimal point, exponent notation, overflow checks, and `strtod` wrappers each require their own basic blocks. `nan` receives few labels because the parsing path is a three-character string comparison.

Per-block accuracy has not yet been reviewed for this output.

5.4. HPACK Decoder

HPACK is the header compression format for HTTP/2, defined in RFC 7541 [6]. The format is fully binary, uses context-sensitive encoding, and includes a fixed Huffman table baked into the specification. This would make HPACK a useful stress test for ParseRE: it has binary encodings and conditional logic that text formats lack. The planned target is `nghttp2`’s HPACK decoder, but the repository does not yet contain a template or output for this evaluation.

5.5. ELF Parser

ELF is the standard binary format on Linux and other Unix-like systems. Unlike the sequential text formats above, ELF is offset-driven: the ELF header contains pointers to program header and section header tables, which in turn point to string tables, symbol tables, and other sections. We target `libelf` from `elfutils`, a widely used library for accessing ELF files.

Corpus layout. Figure 3 shows the layout of a single corpus instantiation. Every corpus member is exactly 1,048 bytes with identical offsets, and varies only in three regions: the ELF header (one variant), the two-entry program header table (four variants that change the second program header’s type field across `PT_NOTE`, `PT_INTERP`, `PT_DYNAMIC`, and `PT_NULL`), and the section region (five variants that toggle which sections contain real data versus zeroed bytes). The Cartesian product yields $1 \times 4 \times 5 = 20$ instantiations and 380 pairwise comparisons.

Results. On `libelf`, ParseRE produces a labeled trace-derived graph with 358 nodes and 354 record-pair edges. Of these, 68 basic blocks receive the `section_header`

One 1048-byte ELF64 file. Offsets fixed across all 20 corpus instantiations.

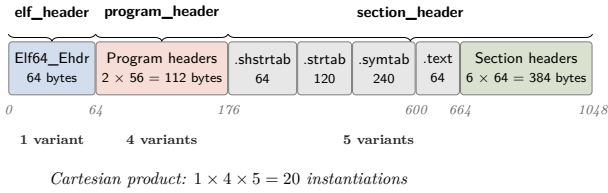


Figure 3. Fixed-layout ELF corpus. Every one of the 20 instantiations has the same byte offsets. The three template children (elf_header, program_header, section_header) carry 1, 4, and 5 alternatives respectively.

label. The program_header production receives zero labels despite having four template variants.

Why program_header is unlabeled. Libelf reads all program-header types through the same code path. The p_type field is data stored in a struct; libelf does not branch on it during parsing. The retained translation-log evidence through gelf_getphdr is therefore identical across the four variants, and the difference algorithm has no record pairs to attribute. Section types, by contrast, do trigger different code paths: sections with SHT_SYMTAB type cause the harness to iterate symbol entries via gelf_getsym and resolve names via elf_strptr, while null sections skip these operations.

TABLE 3. ELF label distribution (libelf). The high ambiguity rate reflects libelf’s structure: most code is shared across section types.

Production	Labeled BBs
section_header	68
ambiguous	202

Where the labels land. Figure 4 shows 12 of the 68 labeled blocks, taken directly from our out.dot. They cluster in three places: the harness’s walk_section_headers loop, libelf’s __libelf_set_rawdata_wlock section-data initialization routine, and libelf’s lookup routines (elf_strptr, gelf_getsym). The cross-function arrows are consecutive records in the QEMU translation log: the corresponding paths appear when the harness reaches libelf’s section setup and symbol-table walk for sections that carry a symbol table.

Accuracy. Manual inspection of all 68 labeled blocks yields 62 strict true positives (91.2%), 5 false positives, and 1 marginal case. The 5 false positives all sit inside __gelf_getehdr_rlock, an ELF-header validation routine that libelf calls internally during section traversal. Those blocks process the ELF header rather than section headers, but they are reached only through section-handling code paths and so appear in the differential edge set. The review did not measure false negatives because it did not independently enumerate every block

12 of the 68 basic blocks labeled section_header by ParseRE

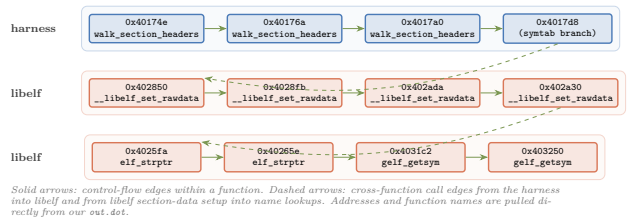


Figure 4. 12 of the 68 section_header-labeled basic blocks. Arrows reproduce consecutive-record relationships from the committed trace-derived graph; dashed styling distinguishes relationships that cross function boundaries.

TABLE 4. Cross-format artifact summary. Inst. = template instantiations; BBs = total blocks in the trace-derived graph; Labels = production-labeled blocks; Acc. = strict true positives divided by all production-labeled blocks.

Format	Parser	Inst.	BBs	Labels	Acc.(%)
URI	Curl	1,458	522	191	—
URI	Apache APR	1,458	—	—	—
JSON	json-c	27	464	295	—
HPACK	nghttp2	—	—	—	—
ELF	libelf	20	358	68	91.2

that should receive a grammar label.

What this reveals about binary formats. The text-format runs produce 6 JSON label strings and 10 exact URI label strings, while the ELF run produces one (section_header). This result is consistent with the tested libelf harness using uniform record readers for many structures and conditional consumer code for section and symbol-table handling. It should not yet be generalized to binary parsers as a class from one evaluated target.

5.6. Cross-Format Comparison

Coverage is not a function of corpus size. JSON uses 27 instantiations and labels 295 blocks (63.6% of the graph). URI uses 1,458 instantiations and labels 191 blocks (36.6%), with 21 additional ambiguous records. ELF uses 20 instantiations and labels 68 blocks (19%). These three observations show that a larger corpus does not automatically yield a higher labeling rate. More formats, parsers, and reviewed outputs are required before attributing the differences to a format class or parser architecture.

6. Related Work

ParseRE occupies a specific niche in the binary-analysis landscape: it associates known input-format labels with a binary’s basic blocks using differential QEMU translation logs. We position it against three lines of prior work that are easily confused with each other but solve materially different problems.

6.1. Trace-Diffing Tools

Tenet [3] and Lighthouse [7] are disassembler plugins that highlight the basic blocks differing between execution traces of the same binary inside IDA Pro or Binary Ninja. They are widely used in practice for tasks like identifying why one input crashes while another does not.

These tools solve a degenerate case of ParseRE’s problem: $N=2$ inputs, no input-side annotations. ParseRE generalizes the differential approach in two ways. First, we systematically generate large corpora of structurally related inputs from a grammar template (Section 3.1) rather than asking the user to supply two interesting inputs by hand. Second, we attach semantic labels to the input structure and propagate those labels to trace-derived record pairs, producing a format-aware annotation rather than a binary “differs / does not differ” verdict.

6.2. Input-Region Categorization

PolyFile [1] from Trail of Bits identifies which byte ranges of a file belong to which substructure of a known file format. PolyTracker [2] extends this with whole-input dynamic information flow tracking, using LLVM instrumentation to build a directed acyclic graph that maps each input byte through the program’s execution.

ParseRE and these tools both operate on inputs whose formats are known, but they differ in what they annotate. PolyFile produces an annotation of the input. PolyTracker produces a data-flow mapping between input bytes and program operations. ParseRE produces candidate annotations for binary locations represented in its trace-derived graph. The three tools are therefore composable: PolyFile can produce the labeled corpus that ParseRE consumes, and ParseRE’s output can be cross-referenced with PolyTracker’s data-flow graph for deeper understanding.

A methodological distinction separates ParseRE from PolyTracker: PolyTracker uses LLVM-level instrumentation, whereas ParseRE observes an executable through QEMU user-mode emulation. The resulting tradeoff is between richer byte-level data flow and a lighter-weight workflow that can be applied when a runnable binary and compatible emulation environment are available.

6.3. Grammar Mining and Format Recovery

A substantial body of academic work addresses the inverse problem: given only a binary, recover the grammar of the inputs it accepts. Tupni [8] performs automatic reverse engineering of input formats using dynamic analysis. Polyglot [9] extracts protocol message formats via dynamic binary analysis. AUTOGRAM [10] infers grammars from observed input-output behavior using dynamic taint analysis. More recent work by

Gopinath et al. [11] mines context-free input grammars from dynamic control flow. Prospex [12] recovers protocol state machines from network traces.

These approaches answer the question “what does this binary eat?” ParseRE answers a different question: “given that this binary eats X, where in the code is X handled?” The two problems are sequential rather than competing. An analyst would use a grammar-recovery tool to obtain a starting grammar, then use ParseRE to map that grammar onto the code. Conflating the two would be a category error.

A second methodological distinction concerns observation cost and evidence. Grammar-recovery systems span compiler instrumentation, dynamic taint analysis, and binary-level execution analysis; several, including Tupni and Polyglot, operate directly on binaries. ParseRE does not recover an unknown grammar or track byte-level data flow. It instead assumes a known template and uses lightweight QEMU translation-log differences, trading semantic depth for easier deployment on compatible runnable binaries.

6.4. Differential Testing for Parsers

The broader idea of detecting parser bugs through differential observation has been explored in the HTTP/1.1 ecosystem [13] and in static differential analysis of protocol parsers [14]. These tools surface discrepancies in protocol semantics between parser implementations. ParseRE surfaces the structural correspondence between an agreed input grammar and the code that realizes it. They are complementary: a differential-testing tool can find that two parsers disagree, then ParseRE can localize where in each parser the disagreement lives.

7. Future Work

Additional architectures. ParseRE currently targets x86-64 binaries. QEMU supports many additional architectures (ARM, MIPS, RISC-V, PowerPC), and the only architecture-dependent component is the trace-log parser. Extending to other architectures requires adapting the regular expression that extracts basic-block boundaries from QEMU’s log output.

Differential fuzzing guided by labels. Once ParseRE has labeled a parser’s CFG, the labels provide a natural coverage metric for format-aware fuzzing. A fuzzer could prioritize inputs that exercise under-explored grammar productions, using ParseRE’s labels as a coverage map.

Grammar mining integration. As discussed in Section 6.3, grammar-recovery tools and ParseRE solve complementary problems. An integrated pipeline could first recover an approximate grammar from a binary, then use ParseRE to validate and refine the mapping, closing the loop between “what does this binary eat?” and “where does it eat it?”

Hierarchical labeling. The current template system supports one level of production-to-label mapping. Extending to hierarchical templates (labeling not just “string” but “string-escape-sequence” versus “string-literal-characters”) would increase the granularity of the output without algorithmic changes.

8. Ethical Considerations

ParseRE is a defensive reverse-engineering tool that annotates binary code with format-semantic labels. It does not discover, create, or exploit vulnerabilities. All evaluated targets are publicly available software projects; the artifact contains no proprietary or classified binaries. The present repository does not include a license file, so it should be treated as source-available for research review rather than as a licensed open-source release.

9. LLM Usage Statement

Large language models were used for editorial assistance and exploratory checks while preparing this draft. The committed ELF label counts reported here were manually reviewed; no unverified model-generated measurement is presented as a final result.

10. Conclusion

We presented ParseRE, a technique for associating known input-format labels with locations in an unknown binary. By systematically generating structurally varied inputs, recording their QEMU translation logs, and associating record-pair deltas with parse-tree deltas, ParseRE produces candidate semantic annotations without requiring source code, compiler instrumentation, or data-flow analysis.

The artifact contains completed runs for URI, JSON, and ELF; manual validation is complete only for ELF, where 62 of 68 labels are strict true positives and one additional label is marginal. These results support continued study, but broader accuracy claims require review of the URI and JSON outputs, completion of the planned APR and HPACK runs, and replacement or validation of the translation-log tracer with dynamic execution events.

References

- [1] E. Sultanik, “PolyFile: A polyglot-aware file identification and mapping utility,” <https://github.com/trailofbits/polyfile>, 2019, trail of Bits; funded by DARPA SafeDocs.
- [2] E. Sultanik, M. Surovič, H. Brodin, K. Kaoudis, F. Tiesca, C. Harmon, L. Overall, J. Sweeney, and B. Larsen, “PolyTracker: Whole-input dynamic information flow tracing,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1841–1845.
- [3] M. Gaasedelen, “Tenet: A trace explorer for reverse engineers,” <https://github.com/gaasedelen/tenet>, 2021, IDA Pro plugin; 1st place, Hex-Rays Plugin Contest 2021.
- [4] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel et al., “SoK: (state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.
- [5] F. Bellard, “QEMU, a fast and portable dynamic translator,” in *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA: USENIX Association, 2005, pp. 41–46.
- [6] R. Peon and H. Ruellan, “HPACK: Header compression for HTTP/2,” RFC 7541, 2015.
- [7] M. Gaasedelen, “Lighthouse: A code coverage explorer for reverse engineers,” <https://github.com/gaasedelen/lighthouse>, 2017, IDA Pro / Binary Ninja plugin; 2nd place, Hex-Rays Plugin Contest 2017.
- [8] W. Cui, M. Peinado, K. Chen, H. J. Wang, and L. Irun-Briz, “Tupni: Automatic reverse engineering of input formats,” in *Proceedings of the 15th ACM Conference on Computer and Communications Security*, ser. CCS ’08. Association for Computing Machinery, 2008, pp. 391–402.
- [9] J. Caballero, H. Yin, Z. Liang, and D. Song, “Polyglot: Automatic extraction of protocol message format using dynamic binary analysis,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007, pp. 317–329.
- [10] M. Höschle and A. Zeller, “Mining input grammars from dynamic taints,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 720–725.
- [11] R. Gopinath, B. Mathis, and A. Zeller, “Mining input grammars from dynamic control flow,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 172–183.
- [12] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, “Prospex: Protocol specification extraction,” in *Proceedings of the 30th IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2009, pp. 110–125.
- [13] B. Jabiyevev, S. Sprecher, K. Onarlioglu, and E. Kirda, “T-Reqs: HTTP request smuggling with differential fuzzing,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’21. Association for Computing Machinery, 2021, pp. 1805–1820.
- [14] M. Zheng, Q. Shi, X. Liu, X. Xu, L. Yu, C. Liu, G. Wei, and X. Zhang, “ParDiff: Practical static differential analysis of network protocol parsers,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 1208–1234, 2024.